

AI Assistant Coding

Lab Assignment-17.1

Name : K.Vyshnavi

H.NO : 2303A51617

Batch : 07

Task 1 – Customer Data Cleaning

Task:

Use AI to generate a Python script for cleaning a customer dataset.

Instructions:

- Handle missing values in columns (age, income, city).
- Remove duplicate records.
- Standardize text values (e.g., city names in lowercase).
- Encode categorical variables (gender, city).

Sample Code:

```
import pandas as pd
data = pd.read_csv("shopping_trends.csv")
print(data.head())
```

Dataset link:

https://www.kaggle.com/datasets/iamsouravbanerjee/customer-dataset?select=shopping_trends.csv

shopping-trends-

- A cleaned Pandas data frame ready for analysis.

Task 1 – Customer Data Cleaning

```
[3] 15s
import pandas as pd

# Upload dataset
from google.colab import files
uploaded = files.upload()

# Load data
data = pd.read_csv(list(uploaded.keys())[0])

print("Original Data:")
print(data.head())

# Handle missing values
data['Age'].fillna(data['Age'].mean(), inplace=True)
# Removed data['income'] as it is not in the dataset
data['Location'].fillna(data['Location'].mode()[0], inplace=True)

# Remove duplicates
data.drop_duplicates(inplace=True)

# Standardize text (lowercase)
data['Location'] = data['Location'].str.lower()

# Encode categorical variables
```

```
[3] 15s
# Encode categorical variables
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()

data['Gender'] = le.fit_transform(data['Gender'])
data['Location'] = le.fit_transform(data['Location'])

print("\nCleaned Data:")
print(data.head())
```

Choose Files shopping_trends.csv

shopping_trends.csv(text/csv) - 453248 bytes, last modified: 4/2/2026 - 100% done
Saving shopping_trends.csv to shopping_trends (1).csv

Original Data:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	\
0	1	55	Male	Blouse	Clothing	53	
1	2	19	Male	Sweater	Clothing	64	
2	3	50	Male	Jeans	Clothing	73	
3	4	21	Male	Sandals	Footwear	90	
4	5	45	Male	Blouse	Clothing	49	

	Location	Size	Color	Season	Review Rating	Subscription Status	\
0	Kentucky	L	Gray	Winter	3.1	Yes	
1	Maine	L	Maroon	Winter	3.1	Yes	
2	Massachusetts	S	Maroon	Spring	3.1	Yes	
3	Rhode Island	M	Maroon	Spring	3.5	Yes	
4	Oregon	M	Turquoise	Spring	2.7	Yes	

	Payment Method	Shipping Type	Discount Applied	Promo Code Used	\
0	Credit Card	Express	Yes	Yes	
1	Bank Transfer	Express	Yes	Yes	
2	Cash	Free Shipping	No	No	

Observation:

1. The dataset initially contained missing values in columns such as *age*, *income*, and *city*, which were handled using mean, median, and mode imputation techniques.

- 2.
3.

Duplicate records were identified and removed, ensuring data uniqueness and consistency.

The *city/location* column had inconsistent text formats (uppercase/lowercase), which were standardized into lowercase for uniformity.
4. Categorical variables like *gender* and *city* were successfully encoded into numerical values, making them suitable for analysis.
5. Data cleaning improved overall data quality and reliability, reducing noise in the dataset.
6. After preprocessing, the dataset became structured and analysis-ready, suitable for visualization and machine learning.
7. The cleaning process ensures that further insights derived from the dataset will be more accurate and meaningful.

Task 2 – Sales Data Preprocessing Task:

Preprocess sales transaction data using AI assistance.

Instructions:

- Convert date columns into datetime format.
- Create new features (month, year, day of week).
- Normalize sales amount column.
- Detect and handle outliers in dataset.

Sample Code:

```
import pandas as pd
data = pd.read_csv("sales_data.csv")
print(data.head())
```

Dataset link:

https://www.kaggle.com/datasets/vinot hkannaece/sales-dataset?select=sales_data.csv

Expected Output:

- A transformed dataset with time-based features and normalized values.

Task 2 – Sales Data Preprocessing

[5] ✓ 22s

```

import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Upload dataset
from google.colab import files
uploaded = files.upload()

data = pd.read_csv(list(uploaded.keys())[0])

print("Original Data:")
print(data.head())

# Convert date column
data['Sale_Date'] = pd.to_datetime(data['Sale_Date'])

# Feature engineering
data['month'] = data['Sale_Date'].dt.month
data['year'] = data['Sale_Date'].dt.year
data['day_of_week'] = data['Sale_Date'].dt.day_name()

# Normalize sales amount
scaler = MinMaxScaler()
data['Sales_Amount'] = scaler.fit_transform(data[['Sales_Amount']])

# Outlier detection (IQR method)
Q1 = data['Sales_Amount'].quantile(0.25)
--

```

[5] ✓ 22s

▶

```

# Outlier detection (IQR method)
Q1 = data['Sales_Amount'].quantile(0.25)
Q3 = data['Sales_Amount'].quantile(0.75)
IQR = Q3 - Q1

data = data[(data['Sales_Amount'] >= Q1 - 1.5*IQR) &
            (data['Sales_Amount'] <= Q3 + 1.5*IQR)]

print("\nProcessed Data:")
print(data.head())

```

⋮

Choose Files

sales_data.csv

sales_data.csv(text/csv) - 105083 bytes, last modified: 4/2/2026 - 100% done

Saving sales_data.csv to sales_data (1).csv

Original Data:

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	\
0	1052	2023-02-03	Bob	North	5053.97	18	
1	1093	2023-04-21	Bob	West	4384.02	17	
2	1015	2023-09-21	David	South	4631.23	30	
3	1072	2023-08-24	Bob	South	2167.94	39	
4	1061	2023-03-24	Charlie	East	3750.20	13	

	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	\
0	Furniture	152.75	267.22	Returning	0.09	
1	Furniture	3816.39	4209.44	Returning	0.11	
2	Food	261.56	371.40	Returning	0.20	
3	Clothing	4330.03	4467.75	New	0.02	
4	Electronics	637.37	692.71	New	0.08	

	Payment_Method	Sales_Channel	Region_and_Sales_Rep
0	Cash	Online	North-Bob

Observation:

2.

3.

1. The *date column* was successfully converted into datetime format, enabling time-based analysis.

New features such as *month, year, and day of the week* were extracted, enhancing the dataset with useful temporal information.

The *sales amount* column was normalized using Min-Max scaling, ensuring all values fall within a standard range (0 to 1).

4. Outliers in the dataset were detected using the IQR method and removed to improve data consistency.
5. Feature engineering improved the dataset by adding valuable insights for trend analysis and forecasting.
6. The processed dataset is now suitable for time series analysis and predictive modeling.
7. Overall preprocessing resulted in a clean, structured, and enriched dataset.

Task 3 – Healthcare Data Preparation Task:

Clean and preprocess a healthcare dataset using AI scripts.

Instructions:

- Handle missing values in *blood_pressure*, *cholesterol*.
- Encode categorical features
- Scale numerical features (min-max or standard scaling).
- Split dataset into training and testing sets.

Dataset link :

<https://www.kaggle.com/datasets/abdallaahmed77/healthcare-risk-factors-dataset>

Expected Output:

- A preprocessed dataset ready for machine learning models.

Task 3 – Healthcare Data Preparation

```
[7]
✓ 26s
import pandas as pd
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
from sklearn.model_selection import train_test_split

# Upload dataset
from google.colab import files
uploaded = files.upload()

data = pd.read_csv(list(uploaded.keys())[0])

print("Original Data:")
print(data.head())

# Handle missing values
data['Blood Pressure'].fillna(data['Blood Pressure'].mean(), inplace=True)
data['Cholesterol'].fillna(data['Cholesterol'].mean(), inplace=True)

# Encode categorical features
le = LabelEncoder()
for col in data.select_dtypes(include='object').columns:
    data[col] = le.fit_transform(data[col])

# Scaling
scaler = MinMaxScaler()
num_cols = data.select_dtypes(include=['int64', 'float64']).columns
data[num_cols] = scaler.fit_transform(data[num_cols])
```

```
[7]
✓ 26s
scaler = MinMaxScaler()
num_cols = data.select_dtypes(include=['int64', 'float64']).columns
data[num_cols] = scaler.fit_transform(data[num_cols])

# Split dataset
X = data.drop('target', axis=1) if 'target' in data.columns else data.iloc[:, :-1]
y = data['target'] if 'target' in data.columns else data.iloc[:, -1]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

print("\nPreprocessed Data:")
print(X_train.head())
```

Choose Files dirty_v3_path.csv
dirty_v3_path.csv(text/csv) - 3379281 bytes, last modified: 4/2/2026 - 100% done
Saving dirty_v3_path.csv to dirty_v3_path (1).csv

Original Data:

	Age	Gender	Medical Condition	Glucose	Blood Pressure	BMI	\
0	46.0	Male	Diabetes	137.04	135.27	28.90	
1	22.0	Male	Healthy	71.58	113.27	26.29	
2	50.0	NaN	Asthma	95.24	NaN	22.53	
3	57.0	NaN	Obesity	NaN	130.53	38.47	
4	66.0	Female	Hypertension	95.15	178.17	31.12	

	Oxygen Saturation	LengthOfStay	Cholesterol	Triglycerides	HbA1c	\
0	96.04	6	231.88	210.56	7.61	
1	97.54	2	165.57	129.41	4.91	
2	90.31	2	214.94	165.35	5.60	
3	96.60	5	197.71	182.13	6.92	
4	94.90	4	259.53	115.85	5.98	

Smoking Alcohol Physical Activity Diet Score Family History \

Observation:

1. Missing values in *blood_pressure* and *cholesterol* were handled using mean imputation, ensuring no data loss.

2.

3.

Categorical variables such as *gender*, *smoking*, and *diabetes* were encoded into numerical format for machine learning compatibility.

Numerical features were scaled using standard scaling, which improves model efficiency and accuracy.

4. The dataset was divided into training and testing sets, allowing proper validation of machine learning models.

5. Preprocessing ensured that all features are in a uniform format, reducing bias during model training.

6. The cleaned dataset is suitable for health risk prediction and classification tasks.

7. Overall, the preprocessing steps enhanced the dataset's quality, usability, and predictive capability.

Task 4 – Employee Salary Data Preprocessing Task:

Clean and preprocess an employee salary dataset using AI scripts.

Instructions:

- Handle missing values
- Detect and remove salary outliers.
- Standardize department names (lowercase).
- Apply label encoding to job location, department.

Dataset link:

<https://www.kaggle.com/code/vishantmathur/employee-salary> Expected Output:

- A structured dataset ready for HR analytics.

Task 4 – Employee Salary Data Preprocessing

```
[13]
✓ 7s
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Upload dataset
from google.colab import files
uploaded = files.upload()

data = pd.read_csv(list(uploaded.keys())[0])

print("Original Data:")
print(data.head())

# Handle missing values
data.ffill(inplace=True)

# Remove salary outliers (IQR)
Q1 = data['Salary(INR)'].quantile(0.25)
Q3 = data['Salary(INR)'].quantile(0.75)
IQR = Q3 - Q1

data = data[(data['Salary(INR)'] >= Q1 - 1.5*IQR) &
            (data['Salary(INR)'] <= Q3 + 1.5*IQR)]

# Standardize department names (removed as 'department' column is not present)
# data['department'] = data['department'].str.lower()
```

```
# Label Encoding (removed as 'job_location' and 'department' columns are not present)
# le = LabelEncoder()
# data['job_location'] = le.fit_transform(data['job_location'])
# data['department'] = le.fit_transform(data['department'])

print("\nFinal Processed Data:")
print(data.head())
```

... Choose Files synthetic_salary_dataset.csv

synthetic_salary_dataset.csv(text/csv) - 28749 bytes, last modified: 4/2/2026 - 100% done
Saving synthetic_salary_dataset.csv to synthetic_salary_dataset (1).csv
Original Data:

	Position	YearsExperience	EducationLevel	Industry
0	NLP Engineer	2.8	PhD	Finance
1	Machine Learning Engineer	12.3	Bachelors	Finance
2	Computer Vision Engineer	4.8	Bachelors	Manufacturing
3	Machine Learning Engineer	1.7	Bachelors	IT
4	MLOps Engineer	12.4	Masters	Healthcare

	Location	Salary(INR)
0	Mumbai	1288000.0
1	Bangalore	1488000.0
2	Remote	897000.0
3	Delhi	676000.0
4	Chennai	2416000.0

Final Processed Data:

	Position	YearsExperience	EducationLevel	Industry
0	NLP Engineer	2.8	PhD	Finance
1	Machine Learning Engineer	12.3	Bachelors	Finance
2	Computer Vision Engineer	4.8	Bachelors	Manufacturing
3	Machine Learning Engineer	1.7	Bachelors	IT

2.

3.

Observation:

1. Missing values in columns like *salary* and *job_location* were handled using appropriate imputation techniques.
2. A significant salary value (outlier) was detected and removed using the IQR method, preventing skewed analysis.
3. The *department* column was standardized into lowercase text, ensuring consistency across records.
4. Categorical features such as *department* and *job_location* were transformed into numerical labels using encoding.
5. Removal of outliers improved the accuracy and reliability of further analysis.
6. The dataset became clean and structured, making it suitable for HR analytics and salary prediction models.
7. The preprocessing pipeline ensures that the dataset is ready for efficient analysis and machine learning applications.